

CS-412 Lab 2: Fuzzing Lib-PNG

Wajiha Masood

Nicolas Sozzi

Sara Bosi

Adrian Martinez

Introduction

Fuzzing is a widely used technique for discovering vulnerabilities in software by automatically generating and mutating inputs to trigger unexpected behavior. In this lab, we apply coverage-guided fuzzing to libpng 1.4.8, a widely deployed open-source library for decoding and encoding PNG images, using AFL++. Given that libpng is embedded in browsers, image editors, and countless other applications, memory corruption bugs in its parsing logic can have significant real-world impact. We describe the design of our fuzzing harness, our instrumentation choices, seed corpus construction, and the analysis of crashes discovered during the campaign, including a heap-based out-of-bounds read in the iTXt chunk parser.

1 Harness Design

We selected `png_read_info()`, `png_read_image()` along with the progressive parser using the `png_process_data()` API as the main entries in the harness. Such APIs make great targets for fuzzing since they are responsible for parsing the user-supplied PNG file and involve complicated operations like handling the chunks in the image, parsing, decompressing the image contents, transforming color values and allocating memory dynamically. The decoders are traditionally one of the main culprits when it comes to memory corruption bugs since the binary format could easily cause an out-of-bounds access, integer overflows or heap corruption.

1.1 Alternative Entry Points

We also explored Libpng write/encoder APIs such as `png_write_image()` and other write APIs, but fuzzing the decoder API was found to be more promising as decoding involves working with user-generated PNG files, whereas encoding works with image data generated by applications. Fuzzing the decoder revealed much more interesting behavior than fuzzing the write API; no crashes were encountered

when fuzzing the latter. Thus, the decoder APIs were chosen as the main target for fuzzing.

1.2 Harness Data flow

The workflow for the harness starts with AFL++ generating a mutated PNG input and passing it to the harness. The input file is then read into a memory buffer that is modeled as a struct that contains data pointers to the input, total size and current offset. With a customized callback function passed to `png_set_read_fn()`, libpng will be able to read directly from the AFL++ controlled buffer instead of from a regular file stream. Through repeated callbacks during execution, libpng will keep requesting bytes, which results in the AFL++ mutated input being sent to the parser. The typical harness involves initializing structures in libpng, reading the PNG header information with `png_read_info()`, reading metadata from the header using `png_get_IHDR()`, activating multiple transform functions, allocating buffers for rows, decoding the PNG images with `png_read_image()`, and lastly parsing additional chunks with `png_read_end()`.

1.3 Guard Rationale

Several guards were included to increase fuzzing robustness and ensure that all crashes occur only because of libpng code rather than harness itself. Checks for null after function calls `png_create_read_struct()` and `png_create_info_struct()` ensure that crashes are not occurring trivially due to failed allocations. It's crucial to include the `setjmp(png_jmpbuf(png))` check because of longjmp based error handling implemented by libpng internally when an invalid PNG file is encountered. Without this mechanism, one bad input can stop fuzzing altogether. Bounds checking inside the custom read function ensures that there will not be any potential out-of-bound reads from the input buffer provided by AFL++. Setting dimensions limits through `png_set_user_limits()` allows for preventing AFL++ from generating huge PNG images with potentially

high memory consumption or denial-of-service attacks. Finally, proper checking and cleanup are specially important in persistent mode since otherwise, memory leaks will be accumulated between successive fuzz runs.

2 Instrumentation and Sanitizers

- **AFL++ Instrumentation (afl-clang-fast):** The target (libpng 1.4.8) was compiled using AFL++ instrumentation with afl-clang-fast compiler flags. This is done in order to instrument the executable so AFL++ can detect execution of any new code path during fuzzing. Without it, AFL++ would mutate inputs mostly blindly and path discovery would be much weaker.
- **AddressSanitizer (ASan):** Address Sanitizer was also used for detecting memory-related bugs such as buffer overflow, out-of-bound access or use after free errors when performing crash analysis to reproduce the bug. In absence of Address Sanitizer, some memory problems can fail to produce a crashable bug or become difficult to debug.
- **Persistent Mode (`__AFL_LOOP()`):** For the persistent-mode campaign, the harness used AFL++’s `__AFL_LOOP()` mechanism. This removes the need to restart the program after every test input, increasing the number of executions per second. The absence of this function will not affect correctness but will slow down the fuzzing process.
- **CRC-32 Patch:** A libpng CRC-32 checksum patch was applied from AFL++’s libpng no-checksum utility. Normally, random input mutations leads to corruption of PNG chunk CRCs and results in file rejection by libpng before parsing takes place. The disabling of CRC checking increases the efficiency of coverage as mutated chunks can bypass this part of code and proceed to other parts.
- **PNG Dictionary and Corpus Minimization:** Finally, PNG dictionary and corpus minimization are used to create a small seed pool that will enable AFL++ to produce PNG files correctly. It is important for efficiency purposes, although absence of these does not hinder the fuzzing process itself.

3 Seed Corpus and Dictionary

We have chosen to use the seed set ‘pngSuite’ from Willem van Schaik [1] in order to have a wide variety of PNGs (with different color-type, bit-depth and interlacing). The seeds are preferably small images to minimize execution time. Since the set was very large, we used the AFL function that minimizes inputs: `afl-cmin -i pngSuite/ -o seeds/ - ./bin/png_fuzz @@`. This command has reduced the number of images from 176 to 80. For the dictionary, we used the one provided by AFL++.¹ Next, in the Figure A.1a, one can see the AFL++ status screen, we see that the dictionary was used 612 k times and led the discovery of 9 new paths

when adding elements. However, it did not find any new paths by inserting words. AFL++ also did not find any new words to add to the dictionary. With the dictionary, we find 15 new paths per million trial. In comparisons, with ‘avoc’, it found 1472 path with 13.8M trials. This gives 107 discoveries per million trials. No splice have been used by the fuzzer.

The dictionary entries are ‘tokens’ or words specific to the domain of the library we are fuzzing (For example for libpng, one can think of the magic word/signature that must be present at the beginning of all PNGs (89 50 4E 47 0D 0A 1A 0A, which spells PNG and adds a bunch of new lines in order to find converting bugs to prevent invalid image as soon as possible). One can also think of the chunks types (e.g. IHDR, IDAT, PLTE). This method increases the likelihood of triggering new behaviors. We save a lot of time compared to only bit inversions, because it would take a long time to produce “89504E470D0A1A0A” in a completely random manner.

4 Campaign Analysis

Firstly, the AFL++ status screen of this campaign shown on Figure A.1a. The stability is at 100%, so the harness is deterministic, which is what we want. The corpus count it at 1828 and as we started with 80 images so the fuzzer found 1748 interesting elements on its own. During the campaign the map reached 33.26%, This means that around 2/3 of the library has never been executed. 57 cycles have been done, so it means that the fuzzer has gone through its entire corpus of inputs 57 complete times, mutating and testing each one.

As visible on Figure B.1a, one can see that the number of edges increases rapidly and reaches a first plateau after around 1 h of execution somewhere between 3200 and 3300 edges. Surprisingly, after around 9 h 30 min, a new region has been discovered where new edges are rapidly found until a second plateau is reached at a bit more than 3400 edges. At the end of the campaign, the last new find was 9 minutes before, so even though fuzzer was slowing down, it was still finding new edges from time to time. If we ran longer, we could have expected some new edges to be found but except if another new region was still accessible but not found, most of the edges that could be discovered with the current harness, has already been found. So with the large number of cycles and the shape of the curve, the campaign has surely reached saturation. Additionally, 238 crashes were saved during the campaign, which will be analyzed in the next section.

5 Crash Triage

A representative crash was selected for triage and replayed with ASan to confirm and analyze the potential bug.

5.1 Step 1: Reproduce

The crash input was copied to a dedicated `triage/` directory and run through `bin/png_fuzz` with ASan symbolization enabled:

```
cp findings/default/crashes/id:000076,...\
  triage/crash_example.png
ASAN_OPTIONS=abort_on_error=0:\
  detect_leaks=0:exitcode=0:symbolize=1 \
  ./bin/png_fuzz triage/crash_example.png

ERROR: AddressSanitizer: heap-buffer-overflow
on address 0x506000000120
READ of size 11 at 0x506000000120 thread T0
#0 strlen (png_fuzz+0x4a3e6)
#1 png_push_read_iTXt  pngpread.c:1606
#2 png_process_data    pngpread.c:41
#3 run_progressive_read harness.c:57
#4 main                harness.c:247

0x506000000120 is located 0 bytes after
64-byte region [0x...e0, 0x...120)
allocated in png_malloc <- png_process_data

SUMMARY: heap-buffer-overflow in strlen
```

ASan reports an out-of-bounds heap read originating from a call to `strlen()` operating on attacker-controlled chunk data. The call chain localizes the fault to libpng's iTXt parsing logic inside `png_push_read_iTXt()`, suggesting that malformed text metadata is processed without sufficient boundary validation.

5.2 Step 2: Minimize with afl-tmin

To isolate the minimal conditions required to reproduce the fault, the original 418-byte crashing input was minimized using `afl-tmin`:

```
AFL_SKIP_CPUFREQ=1 afl-tmin \
-i triage/crash_example.png \
-o triage/crash_min.png \
-- ./bin/png_fuzz @@

Read 418 bytes from 'crash_example.png'.
Program exits with a signal,
minimizing in crash mode.
Block removal complete, 124 bytes deleted.
Symbol minimization: 14 symbols replaced.
Block removal complete, 15 bytes deleted.
Block removal complete, 0 bytes deleted.

File size reduced by : 33.25% (to 279 bytes)
Characters simplified: 125.45%
Number of execs done : 748
```

To validate reproducibility, the minimized input was executed under the ASan-instrumented binary:

```
ASAN_OPTIONS=abort_on_error=0:\
  detect_leaks=0:exitcode=0:symbolize=1 \
  ./bin/png_fuzz triage/crash_min.png

ERROR: AddressSanitizer: heap-buffer-overflow
READ of size 1 at 0x506000000052 thread T0
#0 png_push_read_iTXt  pngpread.c:1581
#1 png_process_data    pngpread.c:41
#2 run_progressive_read harness.c:57

0x506000000052 is located 1 byte after
49-byte region [0x...020, 0x...051)

SUMMARY: heap-buffer-overflow
```

```
pngpread.c:1581 in png_push_read_iTXt
```

The minimized input continues to trigger a heap out-of-bounds read within the iTXt parsing logic, confirming the successful reduction of the input.

5.3 Step 3: Diagnose and Bug Classification

The iTXt parsing logic in `png_push_read_iTXt()` processes multiple variable-length fields stored sequentially and separated by NUL (0x00) delimiters. These fields are traversed using pointer iteration without enforcing strict bounds based on the declared chunk length.

```
/* pngpread.c:1572 */
for (lang = key; *lang; lang++)
    /* Empty loop */ ;

/* pngpread.c:1581 */
for (lang_key = lang; *lang_key; lang_key++)
    /* Empty loop */ ;

/* pngpread.c:1606 */
text_ptr->itxt_length = png_strlen(text);
```

If a malformed iTXt chunk omits a terminator within the allocated region, traversal continues beyond the heap buffer, resulting in a heap-based out-of-bounds read. The issue is similar in class to previously reported libpng text-chunk parsing vulnerabilities (e.g., CVE-2015-8540), but stems from unchecked NUL-terminated traversal rather than integer-related bounds checking. No exact CVE match was identified for libpng 1.4.8.

6 Attack Surface Analysis

6.1 Real-World Applications

Mozilla Firefox. Firefox uses libpng to decode PNG images rendered in web content. PNG data is processed incrementally as it is downloaded over HTTP, and passed to libpng through its progressive decoding interface (`png_process_data()`). This resembles the progressive png decoding strategy used in our fuzzing harness. Because any website can supply arbitrary image data via `<img>` tags, background images, or favicons, the library is exposed to untrusted input in a fully remote, browser-driven context.

GIMP. The GNU Image Manipulation Program relies on libpng to decode and encode PNG files when opening or exporting images. When a user loads an image from an untrusted source (e.g., email attachments or downloaded assets), GIMP parses PNG metadata, including iTXt chunks, using libpng's standard read interfaces. This exposes the same parsing logic exercised in our test harness to user-controlled files in a desktop application context.

6.2 Concrete Attack Scenario

An attacker hosts or distributes a crafted PNG image containing a malformed iTXt chunk. The *Length* field is set to

match the declared keyword size, but the data omits a NUL terminator within the allocated buffer.

When a victim application such as a web browser or image viewer processes the image, libpng parses the file and enters the iTXt decoding routine in `png_push_read_iTXt()`. During this process, pointer-based traversal is used to locate NUL separators between metadata fields. Because no terminator exists within bounds, the parser continues reading beyond the allocated heap buffer. This results in a small out-of-bounds read during string processing (via `strlen()` or equivalent traversal logic), leading to memory access outside the intended allocation.

In practice, this can result in a controlled crash (denial of service) or limited disclosure of adjacent heap memory, depending on the application environment and memory layout. The issue requires no user interaction beyond opening or rendering the image.

6.3 Code Paths Not Exercised by the Harness

Gap 1: Encoder (write) path. The fuzzing harness exercises only read-side PNG processing through `png_process_data()` and does not invoke any write-side APIs such as `png_write_info()` or `png_write_iTXt()`. As a result, the encoder code in `pngutil.c` is not covered.

This matters in applications such as GIMP, where PNG metadata may be read from a file and later written back when exporting an image.

Gap 2: Optional image transform paths. The harness does not enable certain image transformation routines, such as RGB-to-grayscale conversion (`png_do_rgb_to_gray()` in `pngtrans.c`). These paths may be triggered in real-world applications under specific rendering conditions (e.g., grayscale display modes or image filters).

This gap implies that certain execution paths are not explored, and therefore potential vulnerabilities in these components would not be triggered by the current fuzzing campaign.

7 Binary-Only Fuzzing with QEMU Mode

We compared our source-instrumented, ASan-enabled campaign against a binary-only QEMU campaign (`-Q`) compiled without sanitizers or instrumentation. Both used identical versions, harnesses, seeds, dictionaries and CRC patches. Appendix C provides supporting logs and plots.

At 30 minutes, the instrumented vs. QEMU campaigns yielded execution speeds of 313.8 vs. 295.7 exec/s, corpus counts of 1,073 vs. 1,080, and 3,158 vs. 3,648 discovered edges.

Performance: QEMU dynamically translates basic blocks at runtime to inject coverage tracking, which is typically slower than compile-time hooks. However, our 5.8% slowdown was minimal because the baseline’s heavy ASan memory checks offset the penalty of QEMU’s dynamic translation.

Coverage & Crashes: The differing edge counts occur because QEMU reconstructs coverage dynamically from translated blocks, yielding different (and lower-resolution) feedback than precise compiler-inserted hooks. Crucially, the instrumented campaign saved 71 crashes while QEMU saved zero. Without ASan, QEMU only detects fatal OS-level signals (e.g., `SIGSEGV`), entirely missing the silent memory corruptions caught by ASan’s redzones.

8 Instrumentation Depth and Performance

We measured the instrumentation depth at startup using `AFL_DEBUG=1`. To measure the complete library, we linked a minimal driver against `libpng14.a` using `-Wl,-whole-archive`, yielding 14,316 instrumented edges. In contrast, our final harness binary (`bin/png_fuzz`) reported 10,271 edges. The gap comes from how the two binaries were linked. With `-whole-archive`, the measurement binary includes the whole instrumented libpng archive. The final `bin/png_fuzz` binary, however, was linked normally, so it only includes the libpng parts actually needed by the harness. This leaves out unused code, such as encoder-related functions. The harness adds a few extra instrumented edges, but not enough to compensate for the libpng code left out.

For the ASan fork-mode campaign using `bin/png_fuzz`, AFL++ reported 3158 reached edges and a map density of 30.73%. This directly correlates with our harness count (3,158 out of 10,271 equals roughly 30.75%, calculated against the 10,276 target map size). This means that the campaign reached roughly one third of the instrumented edge surface in the final fuzzing binary: the remaining edges represent unexplored code path. Because PNG is highly structured, reaching deeper branches requires precise combinations of chunk types, bit depths, and error states that the fuzzer simply did not trigger during the campaign.

Finally, we compared three execution speeds. The no-sanitizer fork-mode baseline hit 692.06 exec/s, while adding ASan dropped it to 310.50 exec/s (a 2.23x slowdown) due to computationally heavy memory checks and shadow-memory tracking. However, ASan in persistent mode reached 6,199.60 exec/s (a 20x speedup). While fork mode forces the OS to create a new process and execute file I/O for every input, persistent mode completely bypasses this bottleneck by continuously feeding inputs to a single, long-lived target process via a tight `__AFL_LOOP`.

References

- [1] SCHAIK, W. PngSuite — the official set of PNG test images.

Notes

¹AFL++ PNG dictionary: `AFLplusplus/dictionaries/png.dict`

A AFL++ Status Screen Results for the Different modes

american fuzzy lop ++4.41a {default} (bin/png_fuzz) [explore]

process timing	run time : 0 days, 12 hrs, 30 min, 0 sec	overall results
	last new find : 0 days, 0 hrs, 8 min, 37 sec	cycles done : 57
	last saved crash : 0 days, 0 hrs, 4 min, 45 sec	corpus count : 1828
	last saved hang : 0 days, 1 hrs, 49 min, 14 sec	saved crashes : 238
		saved hangs : 1
cycle progress	now processing : 954.215 (52.2%)	map coverage
	runs timed out : 0 (0.00%)	map density : 6.45% / 33.26%
		count coverage : 4.06 bits/tuple
stage progress	now trying : havoc	findings in depth
	stage execs : 76/300 (25.33%)	favorable items : 261 (14.28%)
	total execs : 16.1M	new edges on : 397 (21.72%)
	exec speed : 551.4/sec	total crashes : 451k (238 saved)
		total tmoouts : 679 (0 saved)
fuzzing strategy yields	bit flips : 2/181k, 2/181k, 1/181k	item geometry
	byte flips : 0/22.7k, 0/22.7k, 0/22.6k	levels : 23
	arithmetics : 5/1.59M, 0/3.17M, 1/3.16M	pending : 1
	known ints : 0/203k, 3/860k, 1/1.26M	pend fav : 0
	dictionary : 9/612k, 0/613k, 0/0, 0/0	own finds : 1748
	havoc/splice : 1472/13.8M, 0/0	imported : 0
	py/custom/rq : unused, unused, unused, unused	stability : 100.00%
	trim/eff : 4.80%/963k, 99.81%	
strategy: explore	state: in progress	[cpu007: 25%]

(a)

american fuzzy lop ++4.41a {default} (bin/png_fuzz_persistent) [explore]

process timing	run time : 0 days, 12 hrs, 30 min, 0 sec	overall results
	last new find : 0 days, 0 hrs, 1 min, 36 sec	cycles done : 229
	last saved crash : 0 days, 0 hrs, 6 min, 28 sec	corpus count : 2135
	last saved hang : none seen yet	saved crashes : 304
		saved hangs : 0
cycle progress	now processing : 1823.1583 (85.4%)	map coverage
	runs timed out : 0 (0.00%)	map density : 8.45% / 33.63%
		count coverage : 4.60 bits/tuple
stage progress	now trying : havoc	findings in depth
	stage execs : 129/1200 (10.75%)	favorable items : 251 (11.76%)
	total execs : 174M	new edges on : 404 (18.92%)
	exec speed : 5477/sec	total crashes : 1.21M (304 saved)
		total tmoouts : 14.8k (0 saved)
fuzzing strategy yields	bit flips : 6/56.4k, 1/56.4k, 2/56.3k	item geometry
	byte flips : 0/7046, 0/7034, 0/7010	levels : 37
	arithmetics : 1/492k, 0/981k, 0/977k	pending : 0
	known ints : 1/63.2k, 6/266k, 0/391k	pend fav : 0
	dictionary : 18/190k, 0/190k, 0/0, 0/0	own finds : 2055
	havoc/splice : 1832/169M, 0/0	imported : 0
	py/custom/rq : unused, unused, unused, unused	stability : 99.94%
	trim/eff : 4.31%/1.10M, 99.50%	
strategy: explore	state: in progress	[cpu007: 50%]

(b)

american fuzzy lop ++4.41a {default} (bin/png_fuzz_qemu) [explore]

process timing	run time : 0 days, 12 hrs, 30 min, 0 sec	overall results
	last new find : 0 days, 0 hrs, 12 min, 23 sec	cycles done : 60
	last saved crash : none seen yet	corpus count : 1747
	last saved hang : none seen yet	saved crashes : 0
		saved hangs : 0
cycle progress	now processing : 1382*26 (79.1%)	map coverage
	runs timed out : 0 (0.00%)	map density : 2.14% / 5.76%
		count coverage : 4.10 bits/tuple
stage progress	now trying : havoc	findings in depth
	stage execs : 185/224 (82.59%)	favorable items : 196 (11.22%)
	total execs : 20.3M	new edges on : 319 (18.26%)
	exec speed : 538.7/sec	total crashes : 0 (0 saved)
		total tmoouts : 167 (0 saved)
fuzzing strategy yields	bit flips : 3/78.3k, 1/78.3k, 0/78.3k	item geometry
	byte flips : 0/9786, 0/9774, 0/9750	levels : 25
	arithmetics : 7/684k, 0/1.37M, 0/1.36M	pending : 0
	known ints : 0/88.0k, 1/371k, 3/545k	pend fav : 0
	dictionary : 0/264k, 0/264k, 0/0, 0/0	own finds : 1667
	havoc/splice : 1298/18.1M, 0/0	imported : 0
	py/custom/rq : unused, unused, unused, unused	stability : 100.00%
	trim/eff : 9.40%/724k, 99.82%	
strategy: explore	state: in progress	[cpu007: 25%]

(c)

Figure A.1: AFL++ status screen for the campaign of (a) and (b) the instrumented libpng library, with respectively the persistent was disabled and enabled, (c) the un-instrumented binary (qemu mode). All the campaign durations were 12 h 30 min. All three campaigns have been run simultaneously the same machine on three different CPU, so they were all subject to the same system conditions and are therefore directly comparable in terms of performance.

B Fuzzing Campaign Results

B.1 For ASan and fork mode

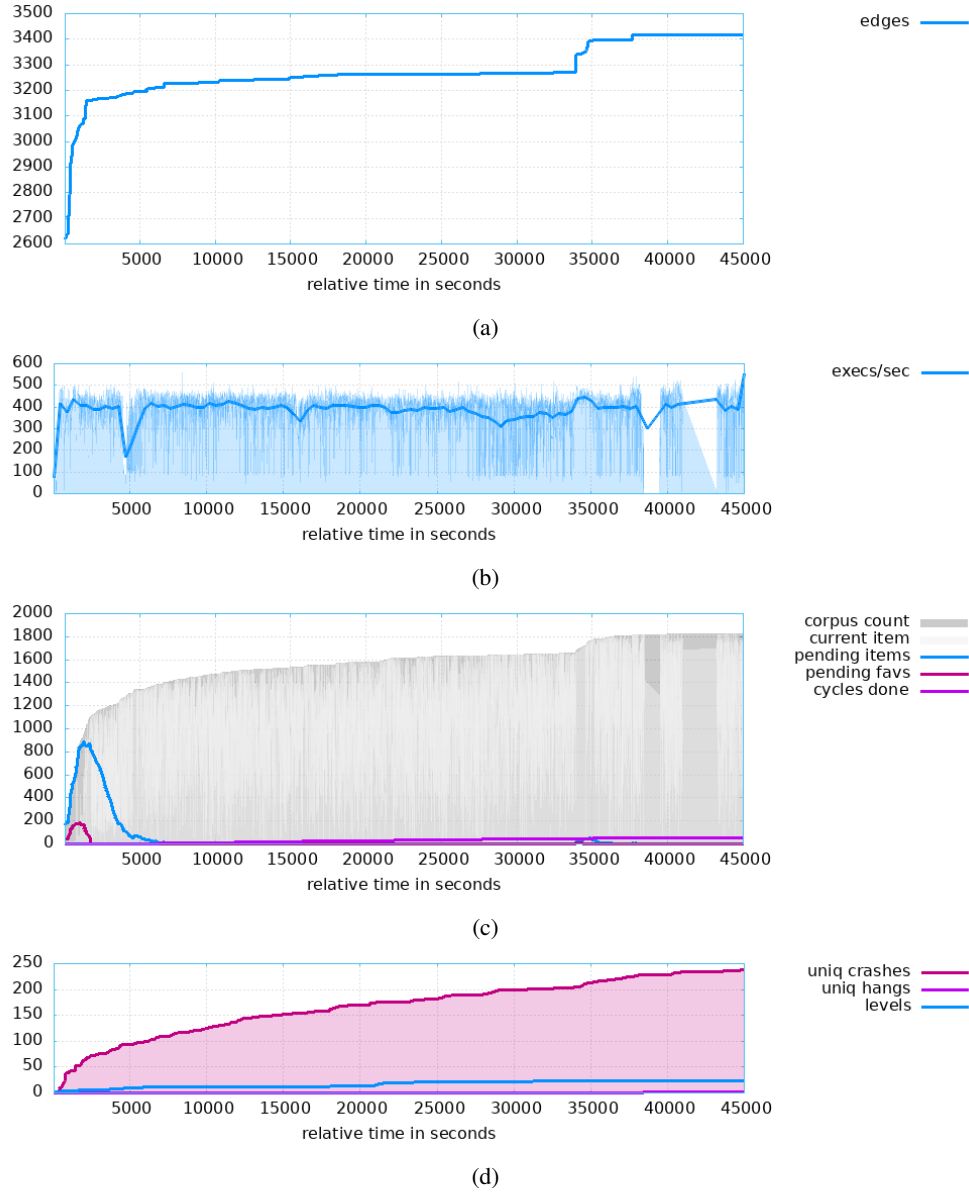


Figure B.1: Afl-plot data analysis of the campaign for the instrumented binary, with fork mode, for different metrics : (a) the number of edges, (b) the execution per second, (c) and (d) time evolution of different elements. The two gaps in the execution per second have been produced by the accidental sleep mode of the computer.

B.2 ASan with Persistent Mode

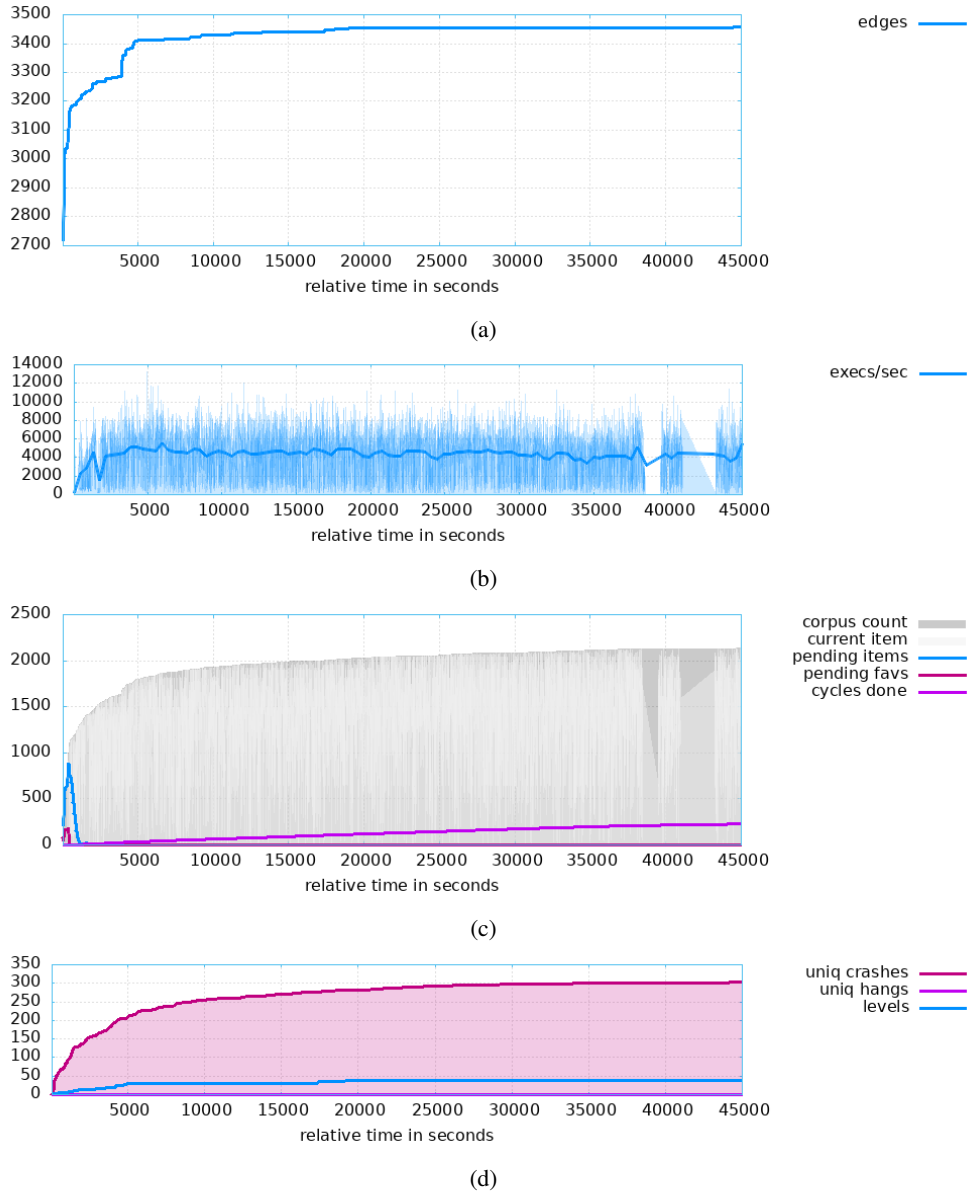


Figure B.2: Afl-plot data analysis of the campaign for the instrumented binary, with persistent mode, for different metrics : (a) the number of edges, (b) the execution per second, (c) and (d) time evolution of different elements. The two gaps in the execution per second have been produced by the accidental sleep mode of the computer.

B.3 Qemu Mode

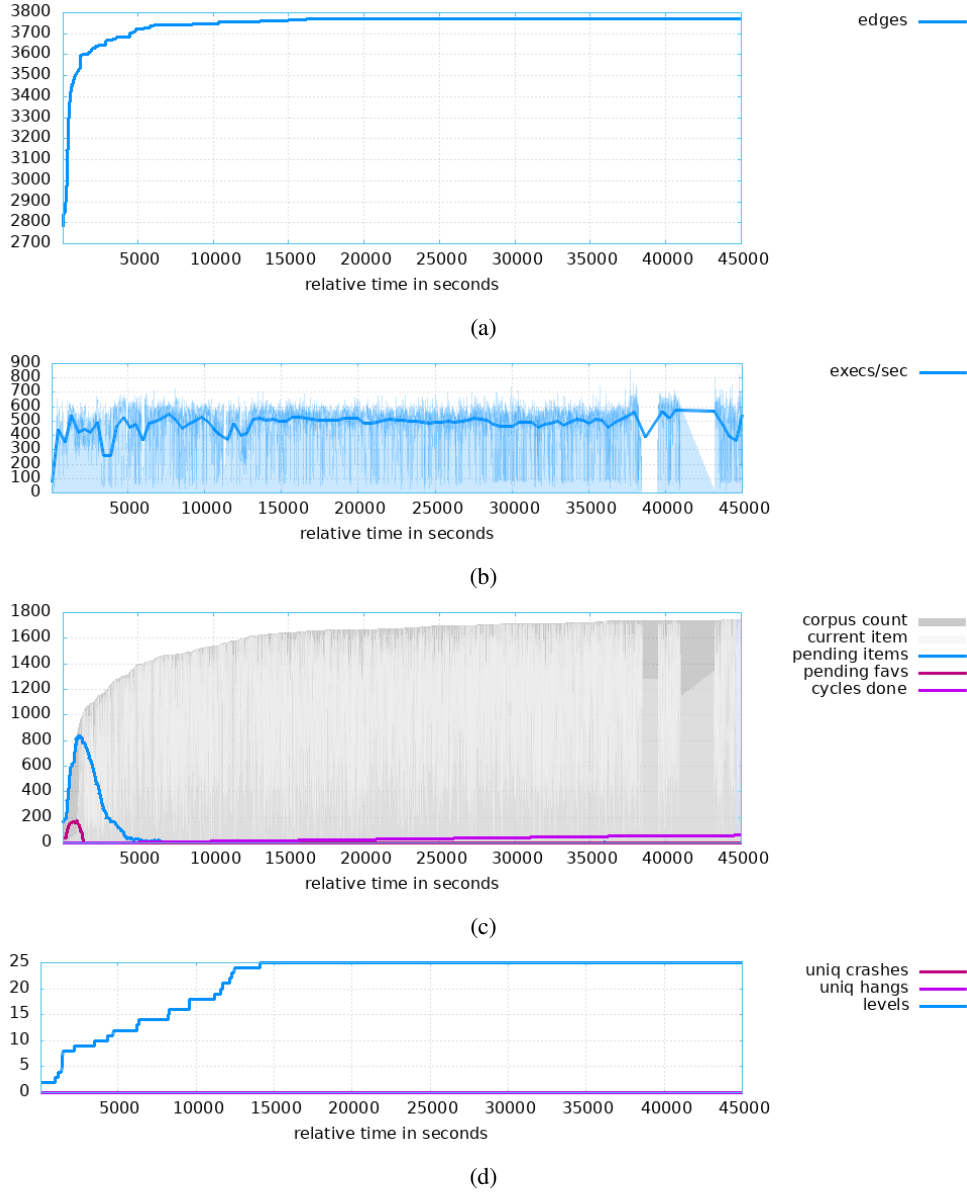


Figure B.3: Afl-plot data analysis of the campaign for the un-instrumented binary (qemu mode), for different metrics : (a) the number of edges, (b) the execution per second, (c) and (d) time evolution of different elements. The two gaps in the execution per second have been produced by the accidental sleep mode of the computer.

C Q7 Campaigns

Table 1 compares the campaigns at the 30-minute mark. Appendix Figures C.1 to C.6 contain the supporting AFL++ status screens and afl-plot graphs.

Metric	Instrumented	QEMU mode
Wall-clock time	1800 sec	1800 sec
Exec speed	313.8 exec/s	295.7 exec/s
Corpus count	1073	1080
Edges found	3158	3648
Overall map density	30.73%	5.57%
Saved crashes	71	0
Total crashes	2152	0
Cycles done	0	0
Stability	100.00%	100.00%

Table 1: Comparison between the source-instrumented campaign and the QEMU-mode campaign after 30 minutes.

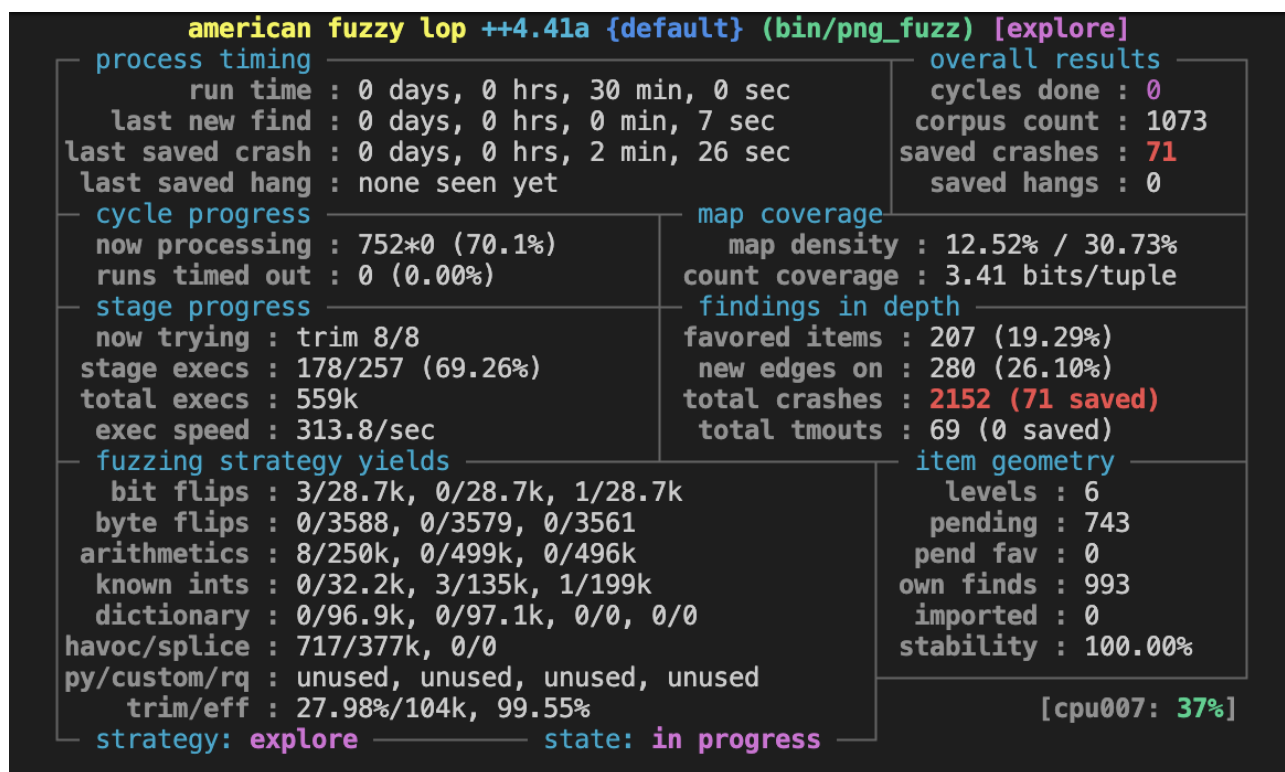
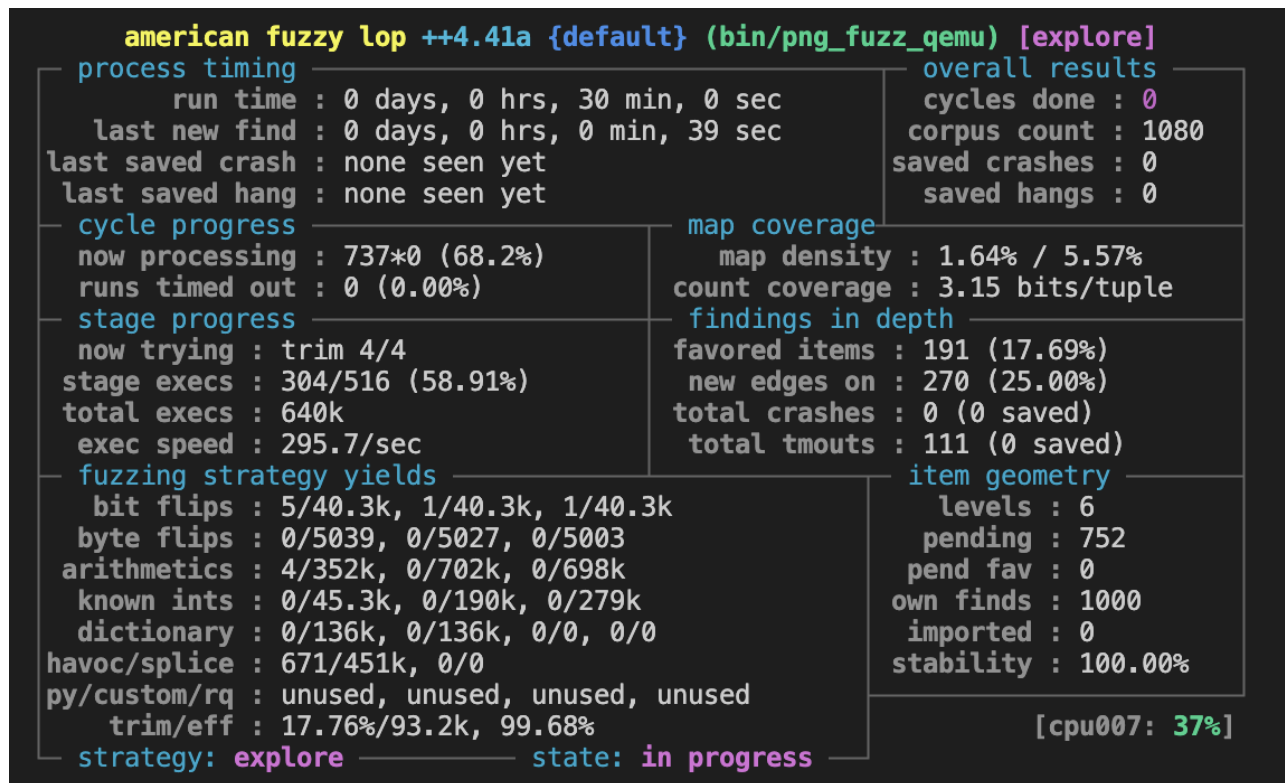
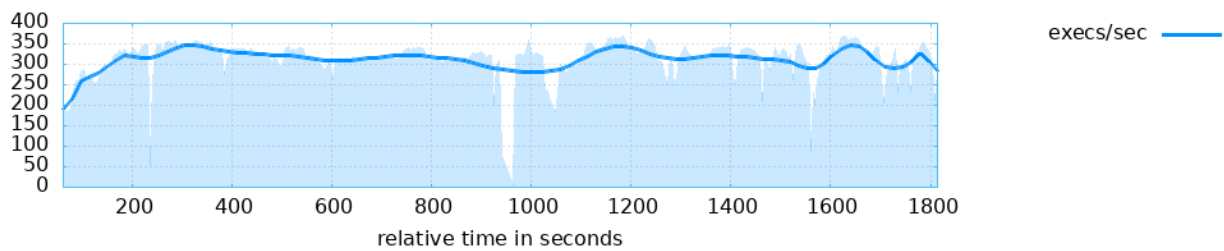
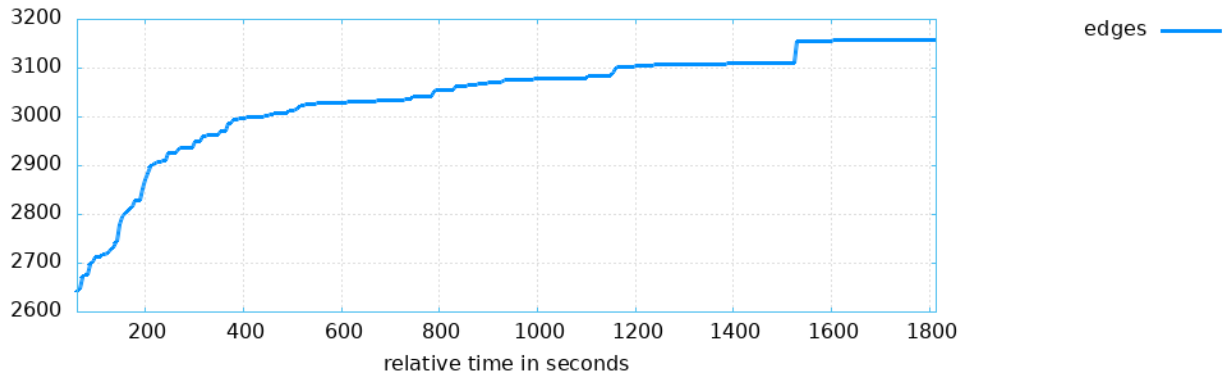


Figure C.1: AFL++ status screen for the source-instrumented campaign after 30 minutes.



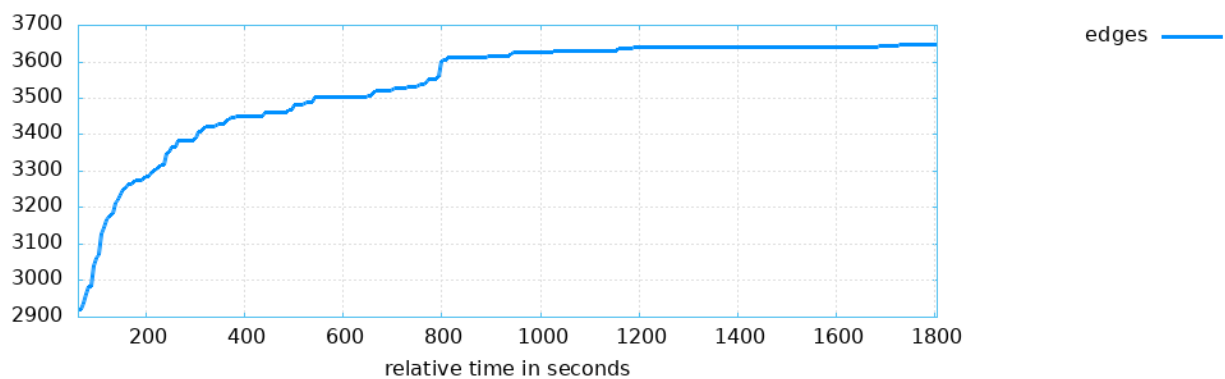


Figure C.5: afl-plot edges graph for the QEMU-mode campaign.

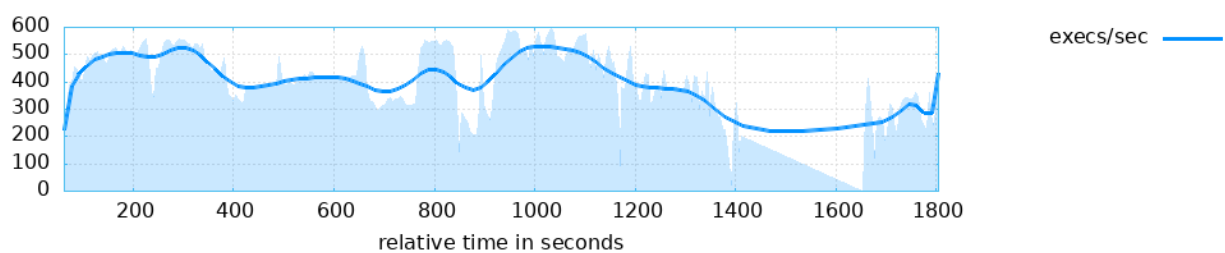


Figure C.6: afl-plot execution speed graph for the QEMU-mode campaign.